

NAVIX

Comment nos modules se parlent

L'idée

On est quatre à bosser chacun sur sa partie du robot. Le problème c'est qu'au bout du compte, faut bien que toutes ces parties se causent. Si mon appli envoie l'adresse de livraison dans un format A et que ton API attend un format B, ça marchera pas.

Ce doc, c'est pour qu'on se mette d'accord avant de coder. Comme ça pas de mauvaise surprise quand on assemblera tout.

J'ai essayé de rester simple. Si certains détails techniques manquent, on en parle en réunion ou on les rajoute après.

Qui fait quoi

Petit rappel pour qu'on parte tous du même point.

Qui	Sa partie	Ce qu'il/elle code
Ninon	L'appli sur le téléphone	Interface client : commander une livraison, suivre le robot en temps réel
Lucas	L'API + MQTT sur le Pi	Reçoit les commandes de l'app, parle au robot, garde tout en base
Léa	Le firmware de l'ESP32	Reçoit les ordres, pilote les moteurs, remonte ce que le robot fait
Alan	La mécanique et l'électronique	Les moteurs, le servo, les capteurs, le câblage

Le chemin d'une commande

Quand un client demande une livraison depuis son téléphone, l'info traverse plusieurs étapes avant que le robot bouge :

```
Téléphone du client (Ninon)
  |
  v par internet (HTTPS + JSON)
API sur le Raspberry Pi (Lucas)
  |
  v par MQTT (en local sur le Pi)
Broker MQTT (Lucas)
  |
  v par MQTT (Wi-Fi)
ESP32 (Léa)
  |
  v par des fils (PWM, GPIO)
```

Pourquoi on a coupé en plusieurs morceaux

Chaque étage ne parle qu'à son voisin direct. Ça nous permet trois choses :

- On peut bosser en parallèle sans dépendre les uns des autres.
- Si on change un truc dans un étage, ça casse pas tous les autres.
- Pour tester, chacun peut simuler son voisin (ex. Léa peut tester son ESP32 avec un faux broker MQTT sans avoir besoin du Pi).

Pourquoi MQTT entre l'API et l'ESP32

On aurait pu faire du REST partout. Mais MQTT c'est plus malin pour un robot :

- C'est très léger, donc l'ESP32 souffre pas.
- C'est asynchrone : l'ESP32 envoie sa position quand il veut, l'API la lit quand elle peut. Personne attend personne.
- Si demain on ajoute un dashboard de monitoring, il s'abonne aux mêmes messages et c'est fini, on touche à rien.

Les règles qu'on se fixe

Règle 1 : tous les messages sont en JSON

Pas de XML, pas de format binaire bizarre. JSON c'est lisible, c'est standard, et toutes nos technos savent le manipuler. Bonus : on peut tout debug avec curl ou mosquitto_sub.

Règle 2 : chaque message a un id, une date et une version

Ces trois champs sont obligatoires dans tous nos messages :

```
{
  "id": "un-id-unique",          // pour pas traiter 2 fois le même
  "timestamp": "2026-06-11T14:32:01Z", // toujours en UTC
  "version": "1.0",             // utile quand on changera le format
  ... le reste dépend du type de message
}
```

L'id sert à éviter qu'une commande soit exécutée deux fois si elle arrive en double (ça arrive avec MQTT). Le timestamp est en UTC pour pas qu'on galère avec les heures d'été. La version, c'est pour le jour où on changera le format des messages — on incrémente et l'ancienne version reste utilisable.

Règle 3 : tous les canaux MQTT s'appellent navix/quelquechose

Les canaux MQTT (qu'on appelle "topics") suivent toujours ce schéma :

```
navix / <type> / <objet>
```

Quelques exemples :

navix/cmd/move	on dit au robot d'aller quelque part
navix/cmd/eject	on lui dit de sortir le colis
navix/state/position	le robot dit où il est
navix/state/battery	le robot dit combien il a de batterie
navix/event/obstacle	quelque chose s'est passé

navix/error/motor un moteur a un problème

Le mot du milieu indique ce que c'est :

- cmd : on donne un ordre au robot (l'API publie, l'ESP32 écoute)
- state : ce que le robot raconte en continu sur son état
- event : un truc qui arrive ponctuellement ("je suis arrivé", "j'ai vu un obstacle")
- error : un pépin

Avantage de cette convention : on peut écouter toute une catégorie d'un coup. Par exemple s'abonner à navix/error/+ pour suivre toutes les erreurs en même temps.

Un exemple concret du début à la fin

Pour rendre tout ça plus parlant, voilà ce qui se passe quand un client demande une livraison.

Les 12 étapes

Étape	Qui	Ce qui se passe
1	App (Ninon)	Envoie POST /delivery à l'API avec l'adresse et le code de retrait
2	API (Lucas)	Vérifie que tout est OK, enregistre la livraison en base
3	API (Lucas)	Publie sur navix/cmd/move
4	ESP32 (Léa)	Reçoit l'ordre et démarre les moteurs
5	ESP32 (Léa)	Publie sa position 5 fois par seconde sur navix/state/position
6	API (Lucas)	Relaie la position à l'appli en temps réel
7	App (Ninon)	Affiche le robot qui bouge sur la carte
8	ESP32 (Léa)	Arrivé à destination : publie sur navix/event/arrived
9	App	Le client tape son code pour ouvrir la trappe
10	API (Lucas)	Vérifie le code et publie sur navix/cmd/eject
11	ESP32 (Léa)	Ouvre la trappe et sort le colis
12	ESP32 (Léa)	Repart à la base tout seul

À quoi ressemble un message concrètement

Petit zoom sur l'étape 3 (l'API balance l'ordre à l'ESP32) :

```
Topic   : navix/cmd/move

Payload :
{
  "id": "cmd-2026-0611-0142",
  "timestamp": "2026-06-11T14:32:02Z",
  "version": "1.0",
  "delivery_id": "del-2026-0611-0142",
  "target": { "lat": 49.0712, "lon": 2.0823 },
  "max_speed_mps": 0.5,
  "timeout_s": 900
}
```

```
}
```

Quand l'ESP32 publie sa position (étape 5), c'est juste l'inverse :

```
Topic   : navix/state/position

Payload :
{
  "id": "pos-19384",
  "timestamp": "2026-06-11T14:32:05Z",
  "version": "1.0",
  "lat": 49.0698,
  "lon": 2.0801,
  "heading_deg": 87,
  "speed_mps": 0.42
}
```

Quand ça plante

On a aussi prévu les cas pas glop. Voici les principaux.

Le Wi-Fi tombe

Le Wi-Fi peut couper à tout moment, c'est la vie. L'ESP32 vérifie sa connexion toutes les 5 secondes. Si elle est coupée, il essaie de se rebrancher avec un délai qui double à chaque fois (1s, 2s, 4s, 8s, max 30s).

Pendant la coupure, les messages que l'ESP32 aurait dû envoyer sont mis en attente (max 10 messages, sinon ça déborde). Et si la coupure dure plus de 10 secondes, le robot s'arrête tout seul par sécurité. Mieux vaut un robot à l'arrêt qu'un robot qui fait n'importe quoi.

Un obstacle bloque le robot

Si l'ultrason détecte un obstacle qui bouge pas, l'ESP32 essaie de contourner (rotation 30°, recul). Au bout de 3 tentatives ratées, il abandonne :

- Il publie une erreur sur navix/error/blocked
- Il allume la LED rouge et fait sonner le buzzer
- Il s'arrête et attend qu'un humain vienne

L'API reçoit l'erreur, marque la livraison comme "en panne" et prévient l'utilisateur via l'appli.

Le robot crashe complètement

Si l'ESP32 plante ou perd l'alim, MQTT a un truc cool qui s'appelle le "testament". Quand l'ESP32 se connecte au début, il dit au broker : si jamais tu me vois plus, envoie ce message d'erreur. Du coup l'API détecte la panne en moins d'une seconde, automatiquement.

Les codes d'erreur qu'on partage

Plutôt que chacun invente ses codes dans son coin, on a une liste commune. La lettre indique la gravité (I = info, W = warning, E = erreur).

Code	Ce que ça veut dire	Qui le balance
I000	OK, une opération a démarré	API ou ESP32
I001	OK, une opération s'est terminée nickel	API ou ESP32
W100	Batterie faible (en dessous de 25%)	ESP32
W101	Obstacle détecté mais on peut contourner	ESP32
E200	La commande reçue est invalide	API
E300	Un moteur est bloqué	ESP32
E301	Le servo force trop, ça craint	ESP32
E400	On a perdu la communication avec le robot	Broker

Ce qu'il reste à faire ensemble

Quelques points sur lesquels j'aimerais qu'on s'aligne dans les prochaines semaines.

Tester tout branché

Avant la soutenance, faut absolument qu'on fasse une session où on connecte tous nos modules en même temps. Un broker MQTT qui tourne, l'API qui répond, l'app qui pointe dessus, le firmware en place. C'est là qu'on verra si nos contrats tiennent vraiment la route. Plus on le fait tôt, mieux c'est.

La sécurité, on verra après

Pour la maquette on reste sur du MQTT en clair sur le réseau local. Si un jour on veut sortir le robot pour de vrai, il faudra activer le chiffrement (MQTT-TLS) et mettre des mots de passe. Pas urgent.

Si quelqu'un change un format

Si l'un de nous touche au format d'un message (rajouter un champ, en virer un, changer le nom d'un topic), on incrémente la version dans le payload et on me prévient. Comme ça je mets à jour ce doc et personne se fait avoir.

Côté debug

Tous les messages MQTT sont sauvés en base par l'API, du coup on aura un historique complet pour debugger après coup. Si on a le temps, on peut faire un petit dashboard web pour voir les échanges en direct — utile en dév et ça en jette en démo.